
LECTURE NOTES

Lecture 10 - Transformers

AI506 — Advanced Machine Learning

Simon Holm
April, 2026



DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1	Recap	3
2	The classic landscape One architecture per “community”	3
3	The Transformer’s takeover	3
3.1	Translation, learned alignment	3
4	Attention is all you need	5
4.1	input embedding	7
4.2	Position Embedding	7
4.3	Attention/self-attention	8
4.4	Multi-headed	9
4.5	Point-wise MLP	9
4.6	Residual connections	9
4.7	Residual connections	10
4.8	LayerNorm	10
4.9	At training time: Masked self-attention	10
4.10	types of Transformer architectures	10

1 Recap

Recap how word embeddings work

- Segment sequence into tokens
- can be optimized with gradient descent

Recap recurrent neural nets

- Powerful sequence models
- ABle to process arbitrary length inputs
- Parameter can be shared sharing across positions
- Drawbacks/Challenges:
 - Long-range dependencies are better, but still challenging

2 The classic landscape One architecture per “community”

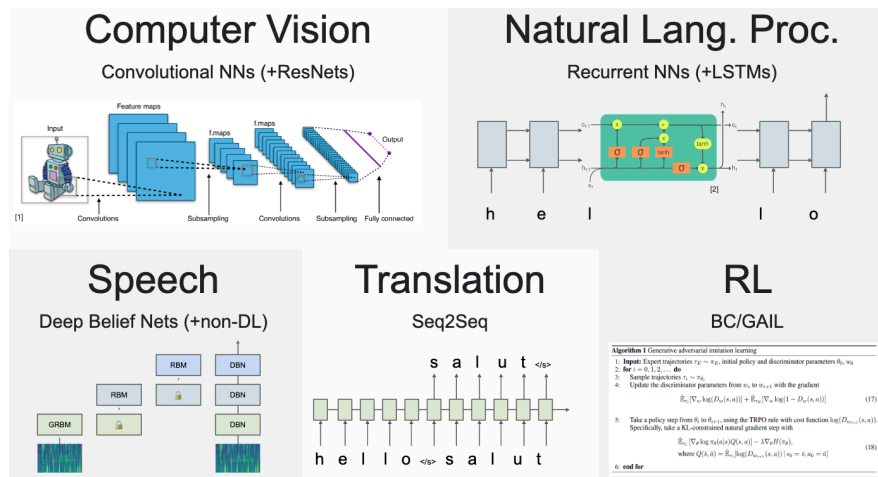


Figure 1: before the transformer, every field had very different architecture

3 The Transformer’s takeover

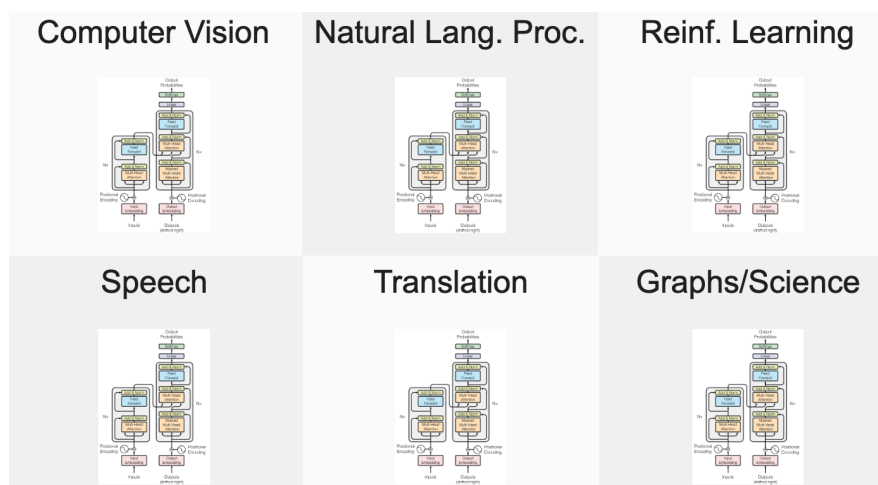


Figure 2: after the transformer, most fields use this

3.1 Translation, learned alignment

Translation can be done in many ways,

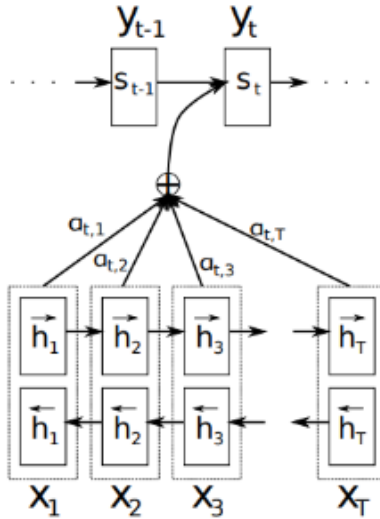


Figure 1: The graphical illustration of the proposed model trying to generate the t -th target word y_t given a source sentence $(x_1, x_2, \dots, x_T)$.

Figure 3: aritechture of translation model, which should generate word y_t by input $\{x_1, x_2, \dots, x_T\}$

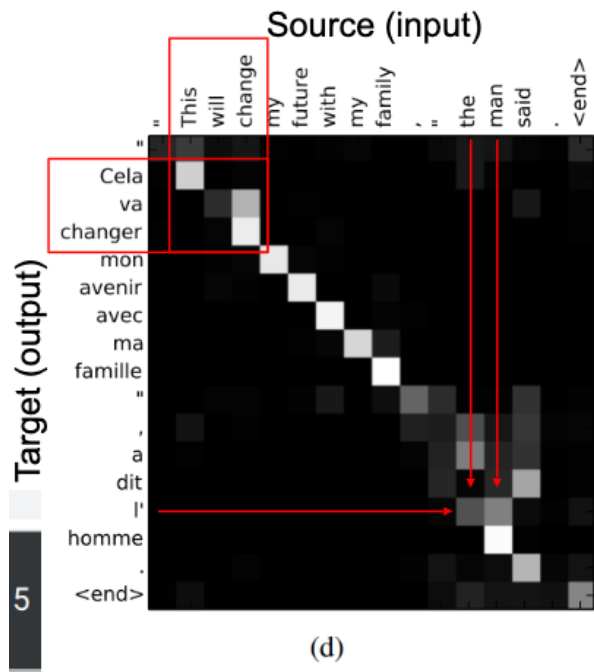


Figure 4: Visualizing this model

4 Attention is all you need

Attention is a function similar to a soft key-value dictionary lookup. The attention is defined as $x = \sum \hat{a}_i v_i$, where each weight, $\hat{a}_i = q \cdot k_i$ typically normalized with

$$\text{softmax}(\hat{a}_i) = \frac{e^{\hat{a}_i}}{\sum_j e^{\hat{a}_j}}$$

Usually, k and v are derived from the same input x :

$$k = W_k \cdot v, \quad v = W_v \cdot x$$

then the query can come from a separate input y (another set). This could be relevant in tasks like translation

$$q = W_q \cdot y$$

Or we can do “self-attention” (within the same set of words)

$$q = W_q \cdot x$$

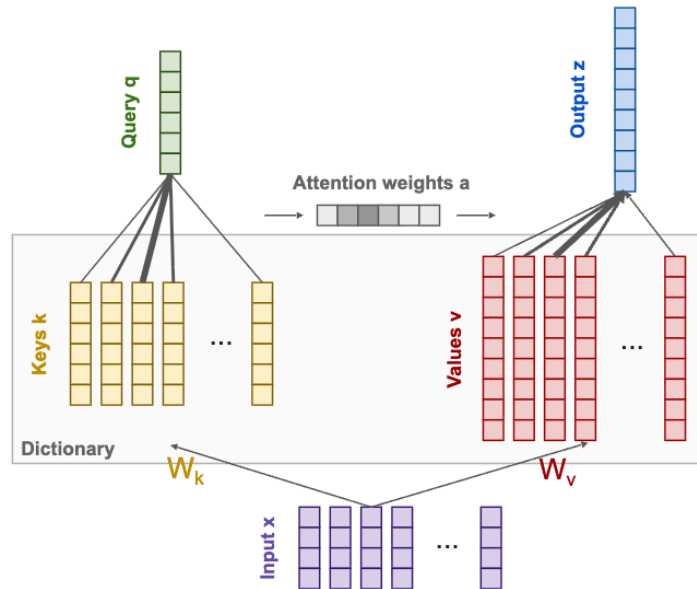


Figure 5: Visualizing the “dictionary” of keys and queries for the output z

We can use many queries, not just one. This can lead to the attention matrix

$$z_{1:M} = \text{Attn}(q_{1:M}, x) = [\text{Attn}(q_1, x) \mid \text{Attn}(q_2, x) \mid \dots \mid \text{Attn}(q_M, x)]$$

We usually use “multi-head”, where we have multiple attentions for each token. This means the operation is repeated K times and the results are concatenated along the feature dimension. We differ

$$z_i = [\text{Attn}_1(q_i, x), \text{Attn}_2(q_i, x), \dots, \text{Attn}_{K(q_i, x)}]$$

The most common see formulation is

$$z = \text{softmax}\left(\frac{QK'}{\sqrt{d_{\text{key}}}}\right) \cdot V$$

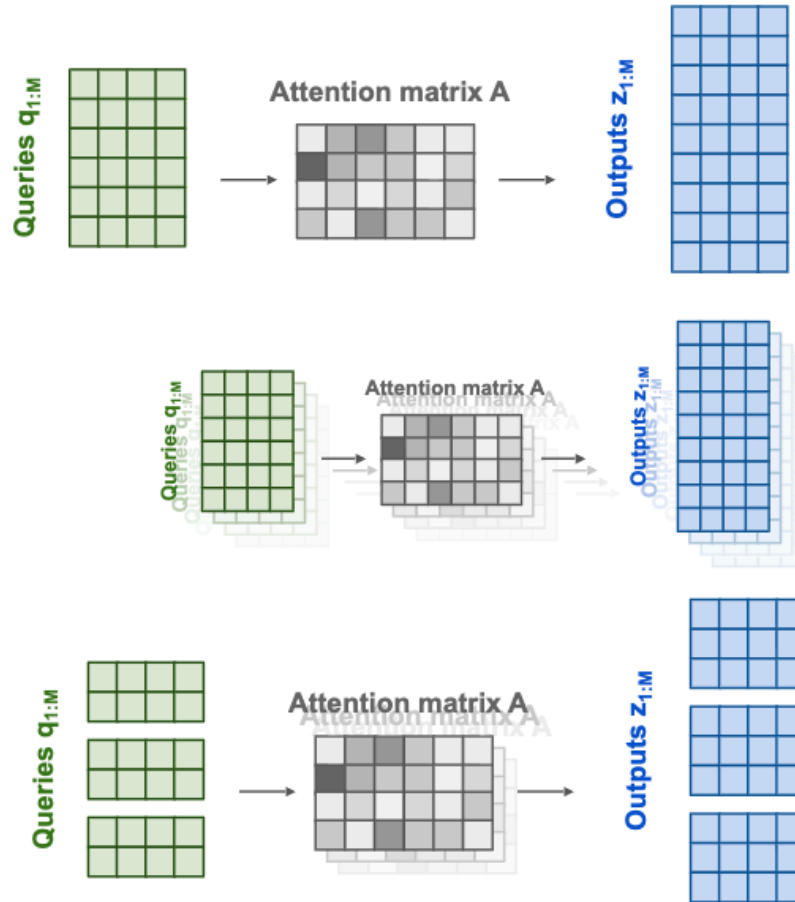


Figure 6: Visualizing the queries in attentions matrices

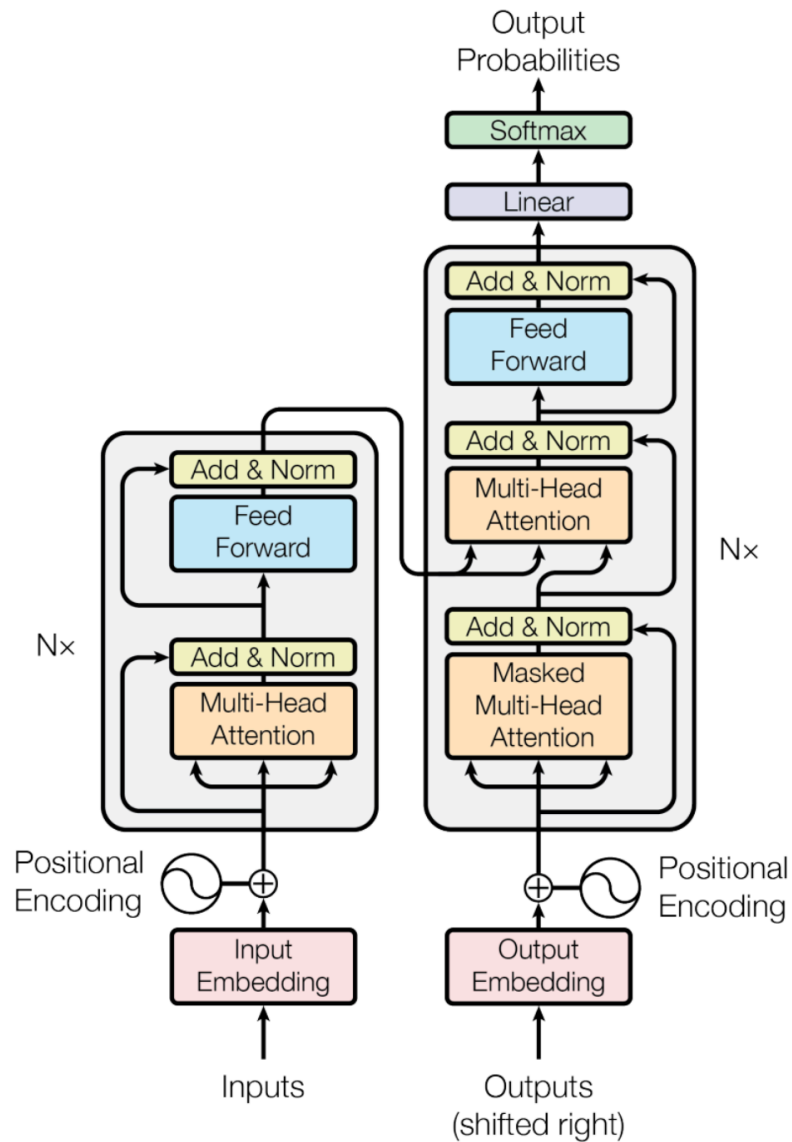


Figure 7: This process can be split up.

- Input Embedding
- Position embedding

4.1 input embedding

Input text is first split into pieces. Can be characters, word, “tokens” (Tokenization)

4.2 Position Embedding

Many different ways of embedding the positions of tokens in the data. We add another embedding table, indexed by positions from 0 (first position) to some maximum n



Figure 8:

4.3 Attention/self-attention

The input sequence is used to create queries, keys, and values! As an example, the formular

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

```
class SelfAttention(nn.Module):
    def __init__(self, d_model, d_key):
        super().__init__()
        self.w_q = nn.Linear(d_model, d_key)
        self.w_k = nn.Linear(d_model, d_key)
        self.w_v = nn.Linear(d_model, d_model)

    def forward(self, x):
        q = self.w_q(x)
        v = self.w_v(x)
        k = self.w_k(x)

        def attention(Q,K,V):
            return F.softmax((Q @ torch.transpose(K, -2, -1))/sqrt(K.size(dim=-1)),
dim=-1) @ V

        return attention(q,k,v)
```


4.4 Multi-headed

More heads = more attention between parameter / more complexity

Example:

```
class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model, d_key, n_heads):
        super().__init__()
        self.heads = nn.ModuleList([SelfAttention(d_model, d_key) for _ in
range(n_heads)])
        self.w_o = nn.Linear(n_heads * d_model, d_model)

    def forward(self, x):
        result = []
        for head in self.heads:
            result.append(head.forward(x))
        result = torch.cat(result, dim=-1)
        return self.w_o(result)
```

4.5 Point-wise MLP

A simple MLP applied to each token individually:

$$z_i = W_2 \text{ GeLU}(W_1 x + b_1) + b_2$$

Think of it as each token pondering for itself about what it has observed previously.

Example in Section 4.6

4.6 Residual connections

For each module, we add the input afterwards, to make shortcuts for the gradient.

$$z_i = \text{Module}(x_i) + x_i$$

Example of both MLP and Residual connection:

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, d_key, n_heads, mlp_factor=4):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = MultiHeadSelfAttention(d_model, d_key, n_heads)
        self.ln2 = nn.LayerNorm(d_model)

        self.mlp = nn.Sequential(
            nn.Linear(d_model, mlp_factor * d_model),
            nn.SiLU(), # Swish activation function, f(x) = x * sigmoid(x)
            nn.Linear(mlp_factor * d_model, d_model)
        )

    def forward(self, x):
        # pre-norm <-- most common
        x = self.attn(self.ln1(x)) + x
        x = self.mlp(self.ln2(x)) + x

        # post-norm
        # x = self.ln1(self.attn() + x)
        # x = self.ln2(self.mlp() + x)
        return x
```

4.7 Residual connections

For each module, we add the input afterwards, to make shortcuts for the gradient.

$$z_i = \text{Module}(x_i) + x_i$$

4.8 LayerNorm

We normalize the current representation of the layer

- there both **post-norm**

$$z_i = \text{LN}(\text{Module}(x_i) + x_i)$$

- and **pre-norm**

$$z_i = \text{Module}(\text{LN}(x_i)) + x_i$$

Example:

```
class TransformerClassifier(nn.Module):
    def __init__(self, n_embeds, n_classes, d_model=256, d_key=64, n_heads=4,
mlp_factor=4, n_layers=2):
        super().__init__()
        self.token_embedding = nn.Embedding(n_embeds, d_model)
        self.transformer_model = nn.Sequential(*[TransformerBlock(d_model, d_key,
n_heads, mlp_factor) for _ in range(n_layers)])
        self.final_layer_norm = nn.LayerNorm(d_model)
        self.classifier = nn.Sequential(nn.Linear(d_model, d_model), nn.SiLU(),
nn.Linear(d_model, n_classes))

    def forward(self, x):
        x = self.token_embedding(x)
        x = self.transformer_model(x)
        x = torch.mean(x, -2)
        x = self.final_layer_norm(x)
        x = self.classifier(x)
        return x
```

4.9 At training time: Masked self-attention

This is regular self-attention as in the encoder, to process what's been decoded so far

training on each $p(z_3|z_2, z_1, x)$, is slow.

Instead we can set zero out all attention weights after each word, so

$$\hat{A} : \text{masked} = A_{\text{raw}} \cdot M, \quad \text{where } M = \begin{pmatrix} 0 & 0 & \dots & 0 \\ 1 & 0 & \dots & 0 \end{pmatrix}$$

4.10 types of Transformer architectures

- Encode only
 - only classification for sentence level task
 - like BERT which can extract information from sentences.
 - “Masked Language Modeling”
 - Cross out some words in a sentence, encoding architecture should fill out the blanks
- Decode-only
 - prediction of tokens
 - “Masked Span Prediction”

- Cross out a sequence of words in a sentence, decoding architecture should fill out the blanks
- Encode-Decode
 - sequence of task (translation)
 - “Next Token Prediction”
 - Predict the next token
 - Called the “Language Modeling Objective”